

AGENTIC CLOUD ASSISTANT (ACA-MCP)

Development Research & Report Requirements

Author	Mihir Menon
Project	ACA-MCP — Agentic Cloud Assistant with MCP
Document Type	Functional Requirements + Academic Report Requirements
Date	7 May 2026
Institution	University Viva Project
Version	1.0

**Abstr
act**

This document serves two purposes. Part A provides complete, self-contained functional requirements, technology specifications, architecture blueprints, and API contracts sufficient to recreate the Agentic Cloud Assistant (ACA-MCP) from scratch. Part B provides structured guidance for writing a 12,000-word academic report on this project, including a literature review template covering cloud automation, AI-driven security assessment, the rise of Large Language Models in cloud operations, identified gaps in the literature, and the contribution of this work.

Table of Contents

PART A — FUNCTIONAL REQUIREMENTS

- 1. System Overview
- 2. Technology Stack
- 3. Architecture & Component Map
- 4. Backend Functional Requirements
 - 4.1 MCP Tool Catalogue (30+ tools)
 - 4.2 AWS Scanning Service
 - 4.3 Security Analysis Engine
 - 4.4 LLM Service (Multi-provider)
 - 4.5 Terraform Generation & Execution
 - 4.6 RAG Knowledge Base
 - 4.7 Cost Analyzer
- 5. Frontend Functional Requirements
 - 5.1 Authentication & Session
 - 5.2 Dashboard Panels
 - 5.3 MCP Client
- 6. API & Protocol Specification
- 7. Data Storage & Persistence
- 8. Deployment Requirements
- 9. Security & Credential Requirements
- 10. Environment Variables

PART B — ACADEMIC REPORT REQUIREMENTS

- 11. 12,000-Word Report Structure
- 12. Literature Review Template
 - 12.1 Cloud Automation — State of the Art
 - 12.2 Cloud Security Assessment & Measurement
 - 12.3 Pre- and Post-LLM Era
 - 12.4 How LLMs Are Used in Cloud Operations
 - 12.5 Limitations & Research Gaps (with Table)
 - 12.6 Gap Addressed by This Work
- 13. Word Count Guidance per Section
- 14. Key Sources & Citation Strategy

PART A — FUNCTIONAL REQUIREMENTS

1. System Overview

The Agentic Cloud Assistant (ACA-MCP) is an AI-powered AWS infrastructure management platform that exposes all capabilities as Model Context Protocol (MCP) tools over HTTP. It enables users to scan AWS resources, detect security misconfigurations, generate and deploy Terraform infrastructure-as-code via LLM, and converse with their live cloud environment in natural language. The same MCP endpoint serves the React web UI, Claude Desktop, and the Claude Code CLI simultaneously.

Core Capabilities

- Live AWS infrastructure scanning (EC2, S3, IAM, Security Groups, VPC)
- Automated 7-rule security analysis with HIGH / MEDIUM / LOW severity ratings
- LLM-powered Terraform HCL generation from plain-English requests
- Human-in-the-loop Terraform plan → approve/reject → apply workflow
- One-click rollback (terraform destroy) of applied infrastructure changes
- Natural-language chat grounded in live AWS account data
- Autonomous security remediation agent (scan → find → fix → plan)
- RAG knowledge base (ChromaDB + SentenceTransformer) for grounded Q&A;
- AWS Cost Explorer integration with LLM cost-optimisation summaries
- Multi-provider LLM routing: Groq (llama-3.3-70b), Anthropic (claude-haiku-4-5), Ollama (local)
- Private key (.pem) generation and secure download after EC2 provisioning
- Full MCP protocol compatibility for Claude Desktop (stdio) and Claude Code CLI (HTTP)

2. Technology Stack

Layer	Technology	Version / Detail	Purpose
Backend Runtime	Python	3.11+	Application runtime
API Framework	FastAPI	Latest	REST API + CORS + lifecycle
MCP Server	FastMCP	Latest	MCP protocol over HTTP / stdio
ASGI Server	uvicorn	Latest	Async server
AWS SDK	boto3	Latest	EC2, S3, IAM, VPC, Cost Explorer APIs
HTTP Client	httpx	Latest	Async calls to LLM APIs
LLM: Groq	groq SDK	Latest	llama-3.3-70b-versatile (default)
LLM: Anthropic	anthropic SDK	Latest	claude-haiku-4-5-20251001

Layer	Technology	Version / Detail	Purpose
LLM: Local	Ollama	HTTP	gpt-oss:120b-cloud (no data egress)
Vector DB	ChromaDB	1.5.7	Persistent embedding store
Embeddings	sentence-transformers	2.7.0	all-MiniLM-L6-v2 (local)
PDF Parsing	PyPDF2	3.0.1	Knowledge base file upload
IaC Engine	Terraform	1.8.5	AWS provisioning via subprocess
Scan Cache	cachetools TTLCache	5-min TTL	Avoid re-scanning on every chat
File Locking	filelock	Latest	Thread-safe execution_log.json
Frontend	React 18 + Vite	Latest	SPA, ES modules
Styling	Plain CSS (CSS vars)	—	No Tailwind dependency
Web Server	Nginx	1.25+	Docker: serves SPA + reverse-proxies backend
Containerisation	Docker + Compose	v2	Two-service: backend + frontend
Config	python-dotenv	Latest	.env file for API keys

3. Architecture & Component Map

The system follows a layered architecture with a single unified entry point (POST /mcp). All business capabilities are MCP tools — the browser UI, Claude Desktop, and Claude Code CLI all speak the same JSON-RPC 2.0 dialect over this one endpoint.

Layer Responsibilities

Layer	File(s)	Responsibility
Browser / MCP Clients	(external)	HTTP JSON-RPC 2.0 tool calls
Nginx (Docker only)	frontend/nginx.conf	SPA serving + reverse proxy + 300 s timeouts
FastAPI App	backend/main.py	Mounts MCP, /health, /terraform/keys/, /rag/documents/upload
FastMCP Server	backend/mcp_server.py	30+ @mcp.tool() definitions in 6 groups
Services Layer	backend/services/	Business logic: scanning, security, LLM, Terraform, cost
RAG Layer	backend/rag/	ChromaDB vector search + prompt augmentation
External APIs	AWS / Groq / Anthropic / Ollama	Cloud data + LLM inference

Key Data Flows

Browser → Backend: `mcpClient.callTool(name, args) → POST /mcp → JSON-RPC 2.0`
`{"method": "tools/call"}` → FastMCP dispatches to `@mcp.tool()` → returns JSON result in SSE or plain JSON

Autonomous Agent Flow: `agent_run(): scan_ec2/s3/iam/sg/vpc → run_security_analysis() → top finding → generate_terraform() → run_terraform_plan() → prompt_llm() summary → frontend awaits human "Approve" → run_terraform_apply()`

RAG Query Flow: `rag_query_tool(question) → ChromaDB cosine search (top-N chunks  $\geq 0.3$ ) → build_augmented_prompt() → prompt_llm() → grounded answer with source attribution`

Chat Flow: `aws_chat(message) → TTLCache[region] (re-scan if expired) → chat_with_groq/anthropic/ollama(message, scan_data, history) → reply string`

4. Backend Functional Requirements

4.1 MCP Tool Catalogue

All tools are decorated with `@mcp.tool()` in `mcp_server.py`. The FastAPI `docs_generator.py` auto-mirrors each as `POST /tools/{name}` for Swagger.

Tool Name	Group	Description	Returns
health_check	A	Verify MCP connection	<code>{"status", "server"}</code>
full_aws_scan	A	EC2+S3+IAM+SG+VPC scan (5 services)	<code>{"ec2", "s3", "iam", "security_groups", "vpc"}</code>
scan_ec2_instances	A	EC2 scan: id, type, state, IPs	<code>{"status", "count", "instances": []}</code>
scan_s3_buckets	A	S3 scan: name, public flag	<code>{"status", "count", "buckets": []}</code>
scan_iam_users	A	IAM: username, MFA, last_login	<code>{"status", "user_count", "users": []}</code>
scan_security_groups_detail	A	SG: id, name, open_to_internet, is_dangerous	<code>{"status", "count", "security_groups": []}</code>
scan_vpc_detail	A	VPC: id, CIDR, is_default, subnet_count	<code>{"status", "count", "vpcs": []}</code>
scan_sg_usage_tool	B	Map every SG → attached ENI resources	<code>{"sg_usage", "unused_sg_ids", "unused_count"}</code>
analyse_security_findings	B	Run 7 rules on raw scan dict	<code>{"total_findings", "severity_counts", "findings": []}</code>
run_security_analysis_with_summary	B	Scan + analyse + LLM summary in one call	<code>{"findings", "severity_counts", "llm_summary"}</code>
estimate_costs	C	Cost Explorer: monthly trend, by service, anomaly	<code>{"current_month", "monthly_trend", "by_service", "anomaly"}</code>
get_cost_with_summary	C	Cost + LLM optimisation summary	<code>{"current_month", "by_service", "llm_summary"}</code>
generate_terraform_hcl	D	HCL from resource_type + config dict	<code>{"hcl", "valid", "validation_message"}</code>

Tool Name	Group	Description	Returns
generate_terraform_from_request	D	HCL from plain-English + existing infra scan	{"hcl","validation","resource_type","description"}
validate_terraform_plan	D	terraform init + validate (no AWS touch)	{"valid","errors","warnings","message"}
run_terraform_plan_mcp	D	Write HCL → init → plan (READ-ONLY)	{"execution_id","status","plan_output","resources_to_*"}
run_terraform_apply_mcp	D	Apply or reject a saved plan (HITL gate)	{"status","apply_output","key_files":[]}
get_execution_history_tool	D	Return all execution log entries	{"executions":[]}
summarise_plan_for_human	D	Parse plan output → plain-English approval text	{"summary","changes_count","risk_level","safe_to_approve"}
aws_chat	E	LLM chat grounded in live scan (TTLCache)	{"reply":str}
agent_run	E	Autonomous: scan→top finding→HCL→plan→summary	{"status","execution_id","issue","hcl","plan_output","summary"}
agent_approve	E	Apply or reject agent-generated plan	{"status","apply_output"}
revoke_open_ingress_rule	E	Direct AWS SDK revoke of 0.0.0.0/0 ingress rule	{"success","revoked","message"}
mark_execution_resolved	E	Mark finding resolved (audit trail)	{"status","execution_id"}
rollback_execution	E	terraform destroy on saved workdir	{"status","destroy_output"}
rag_query_tool	F	ChromaDB search + LLM grounded answer	{"answer","sources","chunks_used"}
rag_list_documents	F	List all knowledge base docs	{"documents":[{"doc_id","chunk_count"}]}
rag_add_text_document	F	Add raw text to ChromaDB	{"chunks_added","doc_id"}
rag_delete_document	F	Delete document + all chunks	{"deleted","message"}
rag_upload_file	F	Add base64 PDF or TXT to ChromaDB	{"chunks_added","doc_id","error"}

4.2 AWS Scanning Service (services/aws_scanner.py)

Each scanner builds its own boto3.Session from credentials dict or env vars. Credential resolution order: (1) explicit dict passed via tool arg → (2) env vars in .env. Both modes are supported simultaneously.

Function	AWS API Called	Data Extracted
scan_ec2(region, credentials)	describe_instances()	InstanceId, InstanceType, State, PublicIpAddress, PrivateIpAddress, LaunchTime, SecurityGroups, Tags

Function	AWS API Called	Data Extracted
scan_s3(credentials)	list_buckets(), get_bucket_acl()	BucketName, CreationDate, is_public (AllUsers ACL grants)
scan_iam(credentials)	list_users(), list_mfa_devices(), get_login_profile()	UserName, UserId, CreateDate, has_mfa, last_login
scan_security_groups(region, credentials)	describe_security_groups()	GroupId, GroupName, Description, VpcId, IpPermissions → open_to_internet (port, protocol, CIDR), is_dangerous
scan_vpc(region, credentials)	describe_vpcs(), describe_subnets()	VpcId, CidrBlock, IsDefault, State, subnet_count
scan_existing_infra_for_context(region, creds)	All 5 scanners combined	Formatted string of existing resource IDs for LLM context injection
scan_sg_usage(region, credentials)	describe_network_interfaces()	SG ID → ENI list → resource type (EC2, RDS, ALB, Lambda, ECS, ElastiCache)
revoke_sg_ingress_rule(sg_id, port, region, creds)	revoke_security_group_ingress()	Revokes all 0.0.0.0/0 + ::/0 ingress rules for the given port

4.3 Security Analysis Engine (services/security_analyzer.py)

Seven deterministic rules applied to scan output. Rules are pure Python — no LLM call. Findings are sorted HIGH → MEDIUM → LOW. Each finding has 9 mandatory fields.

Rule ID	Severity	Triggered When	Resource Type
SSH_PORT_OPEN	HIGH	SG has 0.0.0.0/0 or ::/0 ingress on port 22	EC2_SECURITY_GROUP
RDP_PORT_OPEN	HIGH	SG has 0.0.0.0/0 or ::/0 ingress on port 3389	EC2_SECURITY_GROUP
S3_BUCKET_PUBLIC	HIGH	S3 bucket ACL has AllUsers grant	S3_BUCKET
UNRESTRICTED_ALL_TRAFFIC	HIGH	SG has IpProtocol="-1" from 0.0.0.0/0	EC2_SECURITY_GROUP
IAM_USER_NO_MFA	MEDIUM	IAM user has has_mfa=False	IAM_USER
IAM_USER_INACTIVE	LOW	last_login="Never" OR >90 days ago	IAM_USER
DEFAULT_VPC_IN_USE	LOW	VPC has is_default=True	VPC

Finding schema: finding_id, resource_id, resource_type, rule, severity, title, description, recommendation, metadata (dict). This schema is consumed by both the React UI (severity badge, recommendation text) and the LLM summary generator.

4.4 LLM Service (services/llm_service.py)

Three functions handle provider routing. All calls use httpx.AsyncClient. prompt_llm() is for one-shot tasks (summaries, RAG answers). chat_with_*() functions inject the full scan_data as a JSON system

prompt.

Function	Provider	Model	API Endpoint	Timeout
chat_with_groq(msg, scan_data, history, key)	Groq	llama-3.3-70b-versatile	api.groq.com/openai/v1/chat/completions	30 s
chat_with_anthropic(msg, scan_data, history, key)	Anthropic	claude-haiku-4-5-20251001	api.anthropic.com/v1/messages	30 s
chat_with_ollama(msg, scan_data, history, key)	Ollama (local)	gpt-oss:120b-cloud	localhost:11434/api/chat	180 s
prompt_llm(prompt, model, api_key)	Routes to any of above	Depends on model arg	Same as above per provider	30/180 s

Key design: Anthropic messages use a top-level "system" field (not a role:"system" message). Groq/Ollama inject system context as role:"system" inside messages[]. Temperature 0.3 for chat. max_tokens 1024 for all providers.

4.5 Terraform Generation & Execution

Generation (services/terraform_service.py)

Two generation paths depending on provider. Anthropic uses tool_choice={"type":"tool","name":"generate_terraform"} to force structured JSON output. Groq uses a dual-fenced prompt format (``hcl ... `` + ``json ... ``) parsed with regex. Both produce {"hcl", "resource_type", "description"}. After generation, validate_terraform_syntax() runs terraform init + terraform validate in a temp directory to check HCL correctness before returning to the caller.

Execution (services/execution_service.py)

Each plan/apply gets an isolated directory: terraform_workdirs/exec_{timestamp}_{uuid4[:8]}/. The lifecycle is: (1) write main.tf, (2) terraform init -backend=false, (3) terraform plan -out=tfplan — saved for deterministic apply, (4) on approval: terraform apply -auto-approve tfplan, (5) .terraform/ and tfplan deleted to save disk space, (6) main.tf and .pem key files kept for audit and download. All executions logged to execution_log.json (filelock-protected). Rollback: terraform init + destroy -auto-approve on the saved main.tf.

Command	Flag(s)	Purpose
terraform init	-backend=false -no-color	Download providers (~200 MB hashicorp/aws + hashicorp/random)
terraform plan	-out=tfplan -no-color	Read-only dry-run, saves binary plan for deterministic apply
terraform apply	-auto-approve tfplan -no-color	Apply exactly the saved plan (human already approved)
terraform destroy	-auto-approve -no-color	Rollback: destroy all resources in workdir state
terraform validate	-no-color	Syntax check after HCL generation (in temp dir)

4.6 RAG Knowledge Base (rag/)

- **knowledge_base.py**: chromadb.PersistentClient pointed at chroma_db/ directory (Docker volume). Documents are chunked at 500 chars with 50-char overlap. Embeddings generated locally with sentence-transformers/all-MiniLM-L6-v2 (no external API).
- **rag_service.py**: query_knowledge_base() does cosine similarity search, filters results below relevance threshold 0.3, optionally filters by resource_type, then calls build_augmented_prompt() to embed retrieved chunks before LLM call.
- **seed_knowledge.py**: One-time bulk insert of AWS security documentation into ChromaDB.
- **File ingestion**: POST /rag/documents/upload (multipart) accepts PDF (PyPDF2) or plain text. rag_upload_file MCP tool accepts base64-encoded content for the same operation.
- **Supported resource_type filters**: ec2, s3, iam, vpc, terraform, general.

4.7 Cost Analyzer (services/cost_analyzer.py)

- get_current_month_cost(): boto3 Cost Explorer, DateInterval from 1st of current month to today
- get_monthly_trend(months=N): N months of cost data, one entry per month
- get_cost_by_service(): grouped by SERVICE dimension, returns top-N services
- detect_cost_anomaly(monthly_trend): simple statistical spike detection on trend data
- Cost Explorer is GLOBAL — region parameter is accepted but ignored
- Requires aws ce:GetCostAndUsage IAM permission on the AWS credentials

5. Frontend Functional Requirements

5.1 Authentication & Session (LoginPage.jsx, AuthContext.jsx)

The application uses a simple in-memory authentication gate. Users enter credentials on the LoginPage. AuthContext stores authentication state in React context (not localStorage). The session resets on page refresh — this is intentional for a demo/university project environment.

ApiKeyContext (ApiKeyContext.jsx) persists LLM API keys (Groq/Anthropic) and AWS credentials (access key, secret key, region) across panel re-renders. Keys are passed to tool calls as arguments — no server-side key storage.

5.2 Dashboard Panels (Dashboard.jsx + panels/)

Panel	Component File	MCP Tools Called	Key Features
Infrastructure Overview	InfrastructurePanel.jsx	full_aws_scan	EC2 count, S3 count, IAM user count, SG count, VPC count; metric tiles; live scan button
Security Analysis	SecurityPanel.jsx	run_security_analysis_with_summary	Severity breakdown (HIGH/MEDIUM/LOW), finding list, LLM summary, security score pill

Panel	Component File	MCP Tools Called	Key Features
Security Agent	SecurityAgentPanel.jsx	agent_run, agent_approve, revoke_open_ingress_rule, mark_execution_resolved	Animated 6-step progress, Terraform diff display, Approve/Reject buttons, direct revoke action
Terraform Generator	TerraformPanel.jsx	generate_terraform_from_request, run_terraform_plan_mcp, run_terraform_apply_mcp, rollback_execution	Quick-request pills, HCL display with copy/download, inline execution flow, .pem download
Cost Analysis	CostPanel.jsx	get_cost_with_summary	Monthly spend tiles, cost-by-service chart, anomaly detection, LLM recommendations
Chat	ChatPanel.jsx	aws_chat	Multi-turn chat, model selector (Groq/Anthropic/Ollama), grounded AWS context, history
Knowledge Base	KnowledgeBasePanel.jsx	rag_query_tool, rag_list_documents, rag_add_text_document, rag_delete_document	PDF/text upload, document list, semantic search Q&A;
Execution History	ExecutionHistoryPanel.jsx	get_execution_history_tool	Past plan/apply/destroy log, status badges, audit trail

5.3 MCP Client (src/api/mcpClient.js)

A singleton MCPClient class manages the full JSON-RPC 2.0 lifecycle. On first callTool() it sends an "initialize" handshake with protocolVersion "2024-11-05". After initialize completes it sends a "notifications/initialized" notification. The Mcp-Session-Id header returned by the server is stored and sent on every subsequent request. Responses may be plain JSON or text/event-stream (SSE) — both content types are parsed. On 406 errors the client re-initialises and retries once.

UI/UX Features

- Global CSS custom properties for theming (--accent-blue, --bg, --border, etc.)
- TweaksPanel: runtime accent colour, panel border-radius, density (compact/regular/comfy), monospace font
- SettingsModal: centralised API key + AWS credential entry
- Topbar: navigation, settings button, security score pill (toggleable)
- react-hot-toast notifications for async operation results
- Skeleton loading states (.aca-skel) during API calls
- ErrorBoundary wrapping entire app for graceful crash recovery
- Responsive 12-column CSS grid layout

6. API & Protocol Specification

Primary Endpoint: POST /mcp (MCP JSON-RPC 2.0)

Aspect	Specification
Protocol	JSON-RPC 2.0 over HTTP POST
Content-Type	application/json
Accept	application/json, text/event-stream
Session Header	Mcp-Session-Id (returned by server, sent by client on subsequent calls)
Initialize Method	method: "initialize", params: {protocolVersion, capabilities, clientInfo}
Tool Call Method	method: "tools/call", params: {name: "", arguments: {}}
Response Format	result.content[0].text (JSON string) or SSE data: {...} lines
Error Format	result.isError = true, content[0].text = error message string

Secondary Endpoints (REST only — cannot be MCP due to format constraints)

Endpoint	Method	Purpose	Auth
GET /health	GET	Uptime check — returns {"status":"ok"}	None
GET /terraform/keys/{exec_id}/{filename}	GET	Download .pem private key file (binary)	None (path security by exec_id UUID)
POST /rag/documents/upload	POST	Multipart file upload (PDF or text) to ChromaDB	None
POST /tools/{name}	POST	Auto-generated Swagger test mirror of each MCP tool	None

7. Data Storage & Persistence

Storage	Type	Contents	Docker Volume	Persistence
backend/.env	File	API keys (GROQ_API_KEY, ANTHROPIC_API_KEY, AWS_* vars)	No	Permanent (gitignored)
backend/chroma_db/	ChromaDB	Vector embeddings of knowledge base documents	Yes (chroma_data)	Persistent across container restarts
backend/terraform_workdirs/	Filesystem	Per-execution: main.tf, tfplan, .terraform/, .pem files	Yes (terraform_workdirs)	Persistent; .terraform/ deleted post-apply
backend/execution_log.json	JSON array	Audit log of every plan/apply/reject/rollback	No	In-container (lost on rm)

8. Deployment Requirements

Docker Production Deployment (docker-compose.yml)

- Two services: backend (python:3.11-slim) and frontend (nginx:alpine)
- backend is NOT exposed to the host — only reachable via Nginx proxy
- frontend exposes port 80:80; Nginx serves built React SPA and proxies /mcp, /rag/, /terraform/, /tools/, /health
- Healthcheck: GET http://localhost:8000/health every 30 s; frontend waits for backend healthy
- Nginx timeout 300 s on all proxy routes (Terraform init can take 2-3 min)
- Nginx proxy_buffering off on /mcp route for SSE streaming
- Two named volumes: chroma_data (ChromaDB) and terraform_workdirs (HCL + state)
- Terraform 1.8.5 installed inside the backend image at build time (multi-arch: amd64 + arm64)
- restart: unless-stopped on both services

Local Development Mode

- Backend: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
- Frontend: npm run dev (Vite on :5173)
- Vite proxies /mcp, /rag/, /terraform/ to localhost:8000 (vite.config.js)
- No Nginx required; CORS middleware on FastAPI allows localhost:5173
- Hot reload on both backend (uvicorn --reload) and frontend (Vite HMR)

Claude Desktop / Claude Code MCP Integration

- HTTP mode: add {"url": "http://localhost:8000/mcp"} to MCP config — all 30+ tools available as slash commands
- stdio mode: python mcp_server.py --stdio — Claude Desktop spawns as subprocess, communicates via stdin/stdout
- Both modes use the same FastMCP tool definitions

9. Security & Credential Requirements

- **AWS Credentials:** Two modes supported simultaneously. (1) Env-based: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_DEFAULT_REGION in backend/.env — used for read-only scanning when no credentials passed in tool args. (2) Pass-through: user enters credentials in SettingsModal; forwarded as tool call arguments; used for Terraform apply operations. Credentials are never stored on disk in pass-through mode.
- **LLM API Keys:** GROQ_API_KEY and ANTHROPIC_API_KEY in backend/.env as server-side fallback. Users can supply override keys in the UI — these are passed as api_key argument in each tool call.
- **IAM Permissions Required:** ec2:Describe*, s3:ListBuckets + s3:GetBucketAcl, iam:ListUsers + iam:ListMFADevices + iam:GetLoginProfile, ce:GetCostAndUsage (Cost Explorer), ec2:RevokeSecurityGroupIngress (for direct revoke tool). For Terraform apply: full EC2/S3/IAM/VPC permissions as needed by generated config.

- **PEM Key Security:** Generated RSA 4096-bit keys written to workdir as {name}.pem by Terraform. Download path includes the execution_id UUID as path security. Frontend displays "This is the only time you can retrieve it" warning.
- **CORS:** FastAPI middleware allows only localhost:5173 in development. In Docker, Nginx handles CORS at the proxy level and FastAPI CORS is not the primary gate.

10. Environment Variables (.env)

Variable	Required	Default	Used By
GROQ_API_KEY	Yes (for LLM)	—	llm_service.py, terraform_service.py (groq default)
ANTHROPIC_API_KEY	Optional	—	llm_service.py, terraform_service.py (if model=anthropic)
AWS_ACCESS_KEY_ID	Yes (for AWS)	—	aws_scanner.py, cost_analyzer.py (env fallback)
AWS_SECRET_ACCESS_KEY	Yes (for AWS)	—	aws_scanner.py, cost_analyzer.py (env fallback)
AWS_DEFAULT_REGION	No	us-east-1	boto3 session fallback
OLLAMA_BASE_URL	No	http://localhost:11434	llm_service.py (ollama model)
MCP_TRANSPORT	No	http	mcp_server.py —set to "stdio" for Claude Desktop mode
VITE_MCP_URL	No (Docker)	/mcp	frontend mcpClient.js — relative URL for Nginx proxy

PART B — ACADEMIC REPORT REQUIREMENTS

11. 12,000-Word Report Structure

The following table maps each report section to its recommended word count, key content areas, and the primary claims it must support. Total target: 12,000 words ($\pm 5\%$). Abstract is excluded from the count.

Section	Title	Words	Core Content
Abstract	Abstract	~300	Problem, approach, key results, contribution (not in 12k count)
1	Introduction	800–1,000	Cloud complexity growth; motivation; problem statement; research questions; contributions; report structure
2	Background	600–800	AWS fundamentals; IaC & Terraform; MCP protocol; LLMs in DevOps; RAG systems
3	Literature Review	2,500–3,000	Six subsections (see Section 12): cloud automation, security assessment, pre/post-LLM era, LLM usage, gaps table, this work's contribution
4	System Design	1,200–1,500	Architecture decisions; MCP tool taxonomy; security rule design; LLM provider strategy; RAG pipeline design; human-in-the-loop rationale
5	Implementation	1,800–2,200	Backend deep-dive; FastMCP integration; multi-provider LLM routing; Terraform subprocess lifecycle; ChromaDB chunking; frontend MCP client; Docker deployment
6	Evaluation	1,500–1,800	Security rule accuracy; Terraform HCL validity rate; LLM benchmark (Groq vs Anthropic vs Ollama); RAG relevance scores; agent end-to-end latency; user testing
7	Discussion	1,000–1,200	Gap closure analysis; limitations (single-account, 7 rules, no auth); ethical considerations; unexpected findings
8	Conclusion	600–800	Research question answers; contributions summary; future work (multi-account, GPT-4o, CIS Benchmark full set)
References	References	—	30–50 academic/industry sources; APA/IEEE/Harvard per institution style
Appendices	Appendices	—	Code listings; benchmark raw data; full tool catalogue; .env.example

12. Literature Review Template

The literature review must demonstrate scholarly command of the field and logically build to the gap this project addresses. Each subsection below contains a writing template, key arguments, and recommended sources. Target: 2,500–3,000 words total for the literature review chapter.

12.1 Cloud Automation — State of the Art

Target: ~450 words. Position in report: Section 3.1

Opening claim (write a paragraph like this):

Cloud infrastructure management has evolved through three distinct generations: manual provisioning via web consoles (pre-2010), scripted automation via CLI and configuration management tools such as Puppet and Chef (2010–2014), and declarative infrastructure-as-code (IaC) using Terraform and AWS CloudFormation (2014–present). HashiCorp's Terraform (Mitchell & Bednarski, 2014) introduced a provider-agnostic HCL syntax that became the de facto IaC standard, enabling idempotent, version-controlled cloud resource management. AWS itself formalised automation through CloudFormation stacks and later the CDK (Cloud Development Kit), enabling imperative programming of declarative stacks.

Key arguments to cover:

- The "cloud sprawl" problem: enterprises manage thousands of resources across multiple regions, making manual oversight untenable (Gartner, 2023 cloud management report).
- IaC adoption growth: Terraform downloads exceeded 1 billion by 2023 (HashiCorp State of Cloud report, 2023).
- CI/CD pipeline integration: Terraform Cloud and Atlantis enabled GitOps workflows where infrastructure changes are reviewed like code (pull request model).
- Drift detection: the challenge of configuration drift — when real-world state diverges from declared IaC state — and its security implications.
- Multi-cloud orchestration: tools like Pulumi (imperative) vs. Terraform (declarative) and the trade-offs.
- AWS-native options: CloudFormation, CDK, Systems Manager — and why third-party tools like Terraform often win on portability.
- Key limitation of all pre-LLM automation: users must know exactly what to declare. There is no natural-language interface or self-healing capability.

Recommended sources: HashiCorp (2023); Brikman (2022) "Terraform: Up & Running"; AWS Whitepapers on IaC; Morris (2016) "Infrastructure as Code"; Artac et al. (2017) "DevOps: Introducing Infrastructure as Code".

12.2 Cloud Security Assessment, Measurement & Evaluation

Target: ~500 words. Position in report: Section 3.2

Opening claim:

Cloud misconfigurations have become the primary attack vector in cloud breaches, surpassing exploits as the leading root cause (IBM X-Force Threat Intelligence Index, 2023). Verizon's Data Breach Investigations Report (2023) identified misconfiguration as responsible for 21% of cloud-related breaches. In response, the security community developed structured benchmarks — most notably the CIS AWS Foundations Benchmark — to provide measurable, scored assessments of cloud posture.

Key arguments to cover:

- CIS AWS Foundations Benchmark v3.0 (2023): 58 controls across IAM, Storage, Logging, Networking, Monitoring. The scoring methodology (pass/fail per control, weighted aggregate) established quantitative security measurement.
- AWS Security Hub: native aggregation of findings from GuardDuty, Inspector, Macie, and third-party tools. Introduced the AWS Security Finding Format (ASFF) standard.
- Cloud Security Posture Management (CSPM) tools: Prisma Cloud (Palo Alto), Wiz, Aqua Security. Market grew to \$4.7B by 2024 (MarketsandMarkets). Common approach: agentless API scanning + rule engine.
- Traditional CSPM limitations: findings-only — tools identify problems but leave remediation entirely to humans. No auto-remediation without separate workflow tooling.
- Measurement metrics used in literature: false positive rate, time-to-detect (TTD), time-to-remediate (TTR), severity distribution, benchmark compliance percentage.
- The Capital One breach (2019) as a canonical case study: SSRF exploit exposed AWS metadata, enabled IAM privilege escalation. Root cause: misconfigured WAF + overprivileged role.
- Prior work on automated remediation: AWS Config Rules with auto-remediation SSM documents — limited to pre-defined fixes, no generative capability.

Recommended sources: CIS AWS Foundations Benchmark v3.0 (2023); Verizon DBIR (2023); IBM X-Force (2023); Wiz Cloud Security Report (2024); Fahl et al. (2012) on misconfiguration taxonomy; AWS Security Hub documentation; Dietz & Pernul (2020) on cloud security metrics.

12.3 Pre- and Post-LLM Era in Cloud Operations

Target: ~500 words. Position in report: Section 3.3

Opening claim:

The release of GPT-3 (Brown et al., 2020) and subsequently GPT-4 (OpenAI, 2023) and Claude (Anthropic, 2023) created a categorical shift in cloud tooling. Where pre-LLM cloud operations required practitioners to hold encyclopaedic knowledge of API syntax, IAM policy grammar, and HCL configuration, LLMs enabled natural-language interfaces to infrastructure — a transition analogous to the shift from assembly language to high-level programming.

Pre-LLM era characteristics (before 2022):

- Human expertise bottleneck: only engineers with deep AWS and Terraform knowledge could provision infrastructure.
- Runbook-driven operations: incident response required manually consulting playbooks; no contextual reasoning.
- Static rule engines: CSPM tools applied fixed rules; could not adapt to novel configurations.
- Alert fatigue: high false positive rates in tools like GuardDuty without contextual filtering.
- IaC generation tools: Terraformer (reverse-engineer existing infra to HCL) but no forward generation from intent.

Post-LLM era shifts (2022–present):

- Natural-language IaC generation: ChatGPT, GitHub Copilot, and Amazon Q can draft Terraform/CloudFormation from descriptions.
- Conversational cloud management: products like Amazon Q Developer, Google Gemini in Cloud Console provide chat-based cloud interfaces.

- Contextual security analysis: LLMs can reason about finding severity in the context of the specific workload, not just apply generic rules.
- Agentic operations: early experiments with AutoGPT-style loops for autonomous cloud management (limited production adoption).
- Code understanding: LLMs can explain existing Terraform modules, IAM policies, and CloudFormation templates in plain English.

Recommended sources: Brown et al. (2020) "Language Models are Few-Shot Learners"; Wei et al. (2022) "Emergent Abilities of LLMs"; OpenAI GPT-4 Technical Report (2023); Liu et al. (2023) "Pre-Train, Prompt, and Predict"; GitHub Copilot usage study (Peng et al., 2023); Amazon Q Developer whitepaper (2024).

12.4 How LLMs Are Used in Cloud Operations

Target: ~500 words. Position in report: Section 3.4

Opening claim:

Large Language Models have been integrated into cloud operations along four primary axes: (1) infrastructure code generation, (2) security finding explanation and remediation suggestion, (3) conversational cloud management, and (4) autonomous agentic workflows. Each axis represents a different depth of LLM integration, from augmenting human decisions to autonomously executing infrastructure changes.

Axis 1: Infrastructure Code Generation

LLMs generate Terraform, CloudFormation, CDK, and Kubernetes YAML from natural-language descriptions. Studies show 40–60% of generated IaC requires human correction (Liu et al., 2024). Structured output techniques (tool_choice, JSON mode) significantly improve reliability. Key challenge: LLMs hallucinate resource IDs, ARNs, and region-specific constraints.

Axis 2: Security Finding Explanation

LLMs translate CSPM findings (e.g., "SG allows 0.0.0.0/0 on port 22") into contextual, actionable plain-English recommendations. Amazon GuardDuty now offers LLM-generated summaries. Research by Ferrag et al. (2023) shows LLMs achieve >85% accuracy in classifying cloud misconfigurations.

Axis 3: Conversational Cloud Management

Products like Amazon Q, Microsoft Copilot for Azure, and Google Duet AI provide chat interfaces to cloud consoles. Users can query "how many EC2 instances are running?" and receive accurate answers. Limitation: without live API grounding, LLMs answer from training data (stale by months or years).

Axis 4: Agentic / Autonomous Workflows

ReAct-style agents (Yao et al., 2022) combine LLM reasoning with tool calls. Applied to cloud: agent observes findings, reasons about fixes, executes Terraform, and reports results. Human-in-the-loop (HITL) is essential for production safety. Model Context Protocol (Anthropic, 2024) standardises how agents call external tools.

Recommended sources: Yao et al. (2022) "ReAct: Synergizing Reasoning and Acting"; Nakano et al. (2022) "WebGPT"; Ferrag et al. (2023) "SecurityBERT"; Anthropic MCP specification (2024); Amazon Q Developer (2024); Liu et al. (2024) "LLMs for IaC generation"; Zheng et al. (2023) "Judging LLM-as-a-Judge".

12.5 Limitations & Research Gaps

Target: ~400 words + comparison table. Position in report: Section 3.5

Despite rapid progress, significant gaps remain between the current state of the art and a fully realised AI-powered cloud operations platform. The following table systematically compares existing tools and research against the capabilities provided by this work:

Tool / Work	LLM Integration	Live AWS Scan	Auto-Remediation	MCP Protocol	RAG KB	HITL Gate	Open Source
AWS Security Hub	None (rule-based)	Yes (native)	Via SSM Documents	No	No	No	Managed SaaS
Prisma Cloud (CSPM)	Limited (summaries)	Yes (API polling)	Workflow triggers	No	No	Partial	Commercial
Amazon Q Developer	Full LLM chat	Partial (console)	No	No	No	No	No
GitHub Copilot IaC	Full LLM gen	No (no AWS auth)	No	No	No	No	No
Ferrag et al. (2023)	Classification only	Simulated data	No	No	No	No	Research paper
Liu et al. (2024)	IaC generation	No	No	No	No	No	Research paper
ACA-MCP (this work)	Multi-provider LLM	Yes (live boto3)	Yes (Terraform)	Yes (full)	Yes (ChromaDB)	Yes (approve/reject)	Yes

Key gaps identified from the table:

- **Gap 1 — No unified MCP interface:** No existing tool exposes cloud management as MCP tools, meaning AI agents (Claude, GPT-4) cannot directly control cloud infrastructure through a standard protocol.
- **Gap 2 — No live-grounded LLM chat:** Amazon Q provides chat but not grounded in real-time API data; ACA-MCP scans live AWS before every conversation.
- **Gap 3 — No end-to-end agentic remediation with HITL:** Existing CSPM tools identify findings but require manual remediation; no tool closes the loop from finding → generated fix → validated plan → human approval → applied infrastructure change.

- **Gap 4 — No multi-provider LLM abstraction:** Users are locked to a single LLM provider; ACA-MCP supports Groq, Anthropic, and local Ollama with runtime switching.
- **Gap 5 — No context-aware IaC generation:** Existing LLM IaC generators do not scan the target AWS account before generation, leading to duplicate resource conflicts. ACA-MCP scans existing infra and injects real resource IDs into the generation prompt.

12.6 Gap Addressed by This Work

Target: ~350 words. Position in report: Section 3.6

This work directly addresses Gaps 1–5 through the Agentic Cloud Assistant (ACA-MCP), which makes four primary contributions to the literature:

Contribution 1 — MCP as a cloud operations protocol:

ACA-MCP is, to the author's knowledge, the first system to expose the full lifecycle of cloud infrastructure management (scan, analyse, generate, plan, approve, apply, rollback) as Model Context Protocol tools. This enables any MCP-compatible AI agent to autonomously manage AWS infrastructure through a standardised interface, removing the need for bespoke API integrations.

Contribution 2 — Live-grounded multi-turn cloud conversation:

By performing a live AWS scan before every conversation (with 5-minute TTL caching), ACA-MCP ensures LLM responses are grounded in the actual current state of the cloud account, not training data or stale snapshots. This directly addresses the staleness problem in existing conversational cloud tools.

Contribution 3 — End-to-end agentic remediation with HITL safety gate:

The agent_run() → human approval → terraform apply pipeline demonstrates that fully autonomous cloud remediation is achievable while maintaining safety through a mandatory human-in-the-loop approval step before any infrastructure change is applied. This pattern — AI proposes, human approves, AI executes — is proposed as a reusable design pattern for safe agentic cloud operations.

Contribution 4 — Context-aware IaC generation:

By scanning existing infrastructure before generation and injecting real resource IDs into the LLM prompt, ACA-MCP significantly reduces the rate of duplicate-resource conflicts that plague naive LLM IaC generators. The structured output approach (tool_choice for Anthropic; dual-fenced format for Groq) is evaluated against baseline in the Evaluation chapter.

13. Word Count Guidance per Section

The following targets are recommendations based on the relative importance of each section for a systems-implementation dissertation. Adjust ±15% based on examiner feedback and available evidence.

Section	Min Words	Target Words	Max Words	Priority
1. Introduction	800	950	1,000	High — sets scope

Section	Min Words	Target Words	Max Words	Priority
2. Background	600	700	800	Medium — foundational only
3. Literature Review	2,500	2,750	3,000	HIGH — core academic contribution
4. System Design	1,200	1,350	1,500	High — architecture decisions
5. Implementation	1,800	2,000	2,200	High — technical depth
6. Evaluation	1,500	1,650	1,800	HIGH — quantitative evidence
7. Discussion	1,000	1,100	1,200	High — critical reflection
8. Conclusion	600	700	800	Medium — synthesis
Total	10,000	11,200–12,000	12,500	—

14. Key Sources & Citation Strategy

Aim for 35–50 references. Balance academic papers (>60%), industry reports (~25%), and official documentation (~15%). All claims in the literature review must be cited. Claims in the Implementation section may cite official documentation and GitHub READMEs.

Foundational Academic Papers

- Brown et al. (2020). "Language Models are Few-Shot Learners." NeurIPS. [GPT-3 paper — LLM foundation]
- Wei et al. (2022). "Emergent Abilities of Large Language Models." TMLR. [Why LLMs surprise us]
- Yao et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023. [Agent design]
- Lewis et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS. [RAG foundation]
- Ferrag et al. (2023). "SecurityBERT: A Pre-Trained Transformer for Cybersecurity Text." IEEE Access. [LLM for security]
- Peng et al. (2023). "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." arXiv. [LLM code gen study]
- Dietz & Pernul (2020). "Digital Twin: Empowering Enterprises Towards a System-of-Systems Approach." BISE. [Adapt for cloud digital twin]
- Artac et al. (2017). "DevOps: Introducing Infrastructure-as-Code." ICSE SEIP. [IaC seminal paper]

Industry Reports & Official Documentation

- CIS (2023). "CIS Amazon Web Services Foundations Benchmark v3.0." Center for Internet Security.
- HashiCorp (2023). "State of Cloud Strategy Survey 2023." HashiCorp Inc.
- Verizon (2023). "Data Breach Investigations Report 2023." Verizon Business.
- IBM Security (2023). "IBM X-Force Threat Intelligence Index 2023." IBM Corp.

- Wiz (2024). "The State of Cloud Security 2024." Wiz Inc.
- Anthropic (2024). "Model Context Protocol Specification." docs.anthropic.com.
- OpenAI (2023). "GPT-4 Technical Report." OpenAI.
- AWS (2023). "AWS Well-Architected Framework — Security Pillar." Amazon Web Services.
- Brikman, Y. (2022). "Terraform: Up & Running", 3rd ed. O'Reilly Media.
- Morris, K. (2016). "Infrastructure as Code." O'Reilly Media.

Citation Tips

- Use Google Scholar to verify publication year and journal name before citing.
- For AWS documentation, cite the specific page title and access date (web resource format).
- The MCP specification: cite Anthropic (2024) with URL <https://modelcontextprotocol.io>
- For Terraform 1.8.5, cite the HashiCorp release notes or changelog page.
- If your institution requires IEEE: use [1] numeric superscripts and list in order of appearance.
- If APA: (Author, Year) in-text and alphabetical reference list at the end.
- Avoid citing Wikipedia, Stack Overflow, or blog posts as primary sources in the Literature Review.

Document End

This document (devresearch.pdf) was auto-generated from the live ACA-MCP codebase. All functional requirements reflect the actual implementation as of 7 May 2026. Academic report guidelines are recommendations — verify with your supervisor.